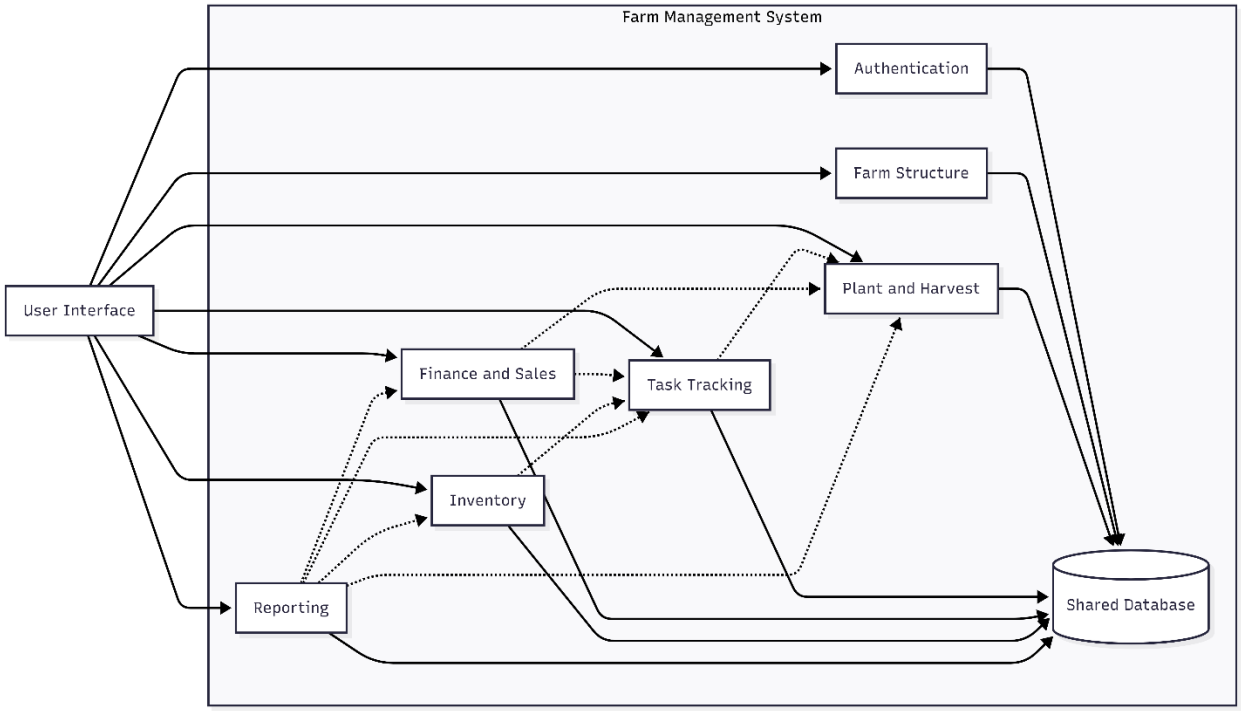
System Design and Architecture

1. Architectural Style Selection

The chosen architectural style for the Farm Management System is a **layered modular monolith**. This style fits the project because the system is divided into clear functional modules that are strongly related to the database structure: user management, farm structure, plant management, task management, harvest/resource tracking, finance and sales, inventory, and reporting. These modules can remain separate in the codebase while still sharing one application and one relational database. This makes the system easier to build, test, and maintain for a small academic project, while still keeping the design organized. The schema itself also supports this choice because the tables are grouped into logical domains with clear relationships, for example farms, plots, greenhouses, and plants for farm structure; tasks, task_assignments, task_comments, and task_history for workflow; and expense_records, sales_deals, sales_deliveries, finance_transactions, inventory_items, and inventory_movements for finance and stock control.

A rejected alternative was **microservices architecture**. Although microservices would separate each subsystem into independent deployable services, that approach would be too heavy for this project. It would introduce unnecessary complexity in deployment, communication, security, and data consistency, especially since the system uses one shared database design with many relationships across modules. For example, tasks are linked to plants, resource usage, workers, expenses, and reports, while sales and finance are also tied back to harvests and farm records. Splitting this into multiple services would increase development overhead without giving a practical benefit for a small system. For this reason, a layered modular monolith is the more appropriate and realistic choice.

2. Component Diagram



2.1 Overview

The Component Diagram should show the Farm Management System as a set of major subsystems that follow the functional grouping already visible in the database structure. Each subsystem should expose a clear interface and interact with the database through its own data access logic. The design should reflect one integrated application divided into modules rather than multiple separate services.

2.2 Major Subsystems

The main components of the system are:

- **Authentication and User Management Component**
- **Farm Structure Management Component**
- **Plant and Harvest Management Component**
- **Task and Work Tracking Component**
- **Finance and Sales Management Component**
- **Inventory Management Component**

- **Reporting and Dashboard Component**
- **Shared Database Component**

2.3 Component Responsibilities

Authentication and User Management Component

This component handles:

- login and session access
- user role handling
- manager and worker account data
- account activation status

It is based mainly on:

- users

Suggested provided interface:

- IUserAuthService
- IUserManagementService

Farm Structure Management Component

This component handles:

- farm creation and updates
- plot management
- greenhouse management
- growing area overview

It is based mainly on:

- farms
- plots
- greenhouses

Suggested provided interface:

- IFarmManagementService

- IGrowingAreaService

Plant and Harvest Management Component

This component handles:

- plant records
- plant stages
- plant status updates
- harvest recording
- resource usage tracking linked to plant work

It is based mainly on:

- plant_stages
- plants
- harvest_history
- resource_usage

Suggested provided interface:

- IPlantService
- IHarvestService
- IResourceUsageService

Task and Work Tracking Component

This component handles:

- recurring task series
- task creation and updates
- worker assignments
- task comments
- task history
- progress tracking
- completion confirmation

It is based mainly on:

- recurring_task_series
- tasks
- task_assignments
- task_comments
- task_history

Suggested provided interface:

- ITaskService
- ITaskAssignmentService
- ITaskTrackingService

Finance and Sales Management Component

This component handles:

- expense categories
- recurring expenses
- manual expense records
- sales deals
- sales deliveries
- finance transactions

It is based mainly on:

- finance_partners
- expense_categories
- recurring_expenses
- expense_records
- sales_deals
- sales_deliveries
- finance_transactions

Suggested provided interface:

- IExpenseService
- ISalesService
- IFinanceTransactionService

Inventory Management Component

This component handles:

- inventory categories
- inventory item records
- stock level tracking
- movement history
- waste and adjustment records

It is based mainly on:

- inventory_categories
- inventory_items
- inventory_movements

Suggested provided interface:

- IInventoryService
- IStockMovementService

Reporting and Dashboard Component

This component handles:

- dashboard summaries
- monthly and yearly reporting
- task summaries
- plant summaries
- financial summaries
- inventory summaries

It reads across multiple modules, especially:

- plants
- tasks
- harvest_history
- expense_records
- sales_deliveries
- finance_transactions
- inventory_items

Suggested provided interface:

- IReportingService
- IDashboardService

Shared Database Component

This component stores all persistent data and supports all functional modules.

It contains the full relational schema, including:

- users
- farms and growing areas
- plants and plant stages
- tasks and work tracking tables
- harvest and resource usage tables
- finance and sales tables
- inventory tables

2.4 Component Interactions

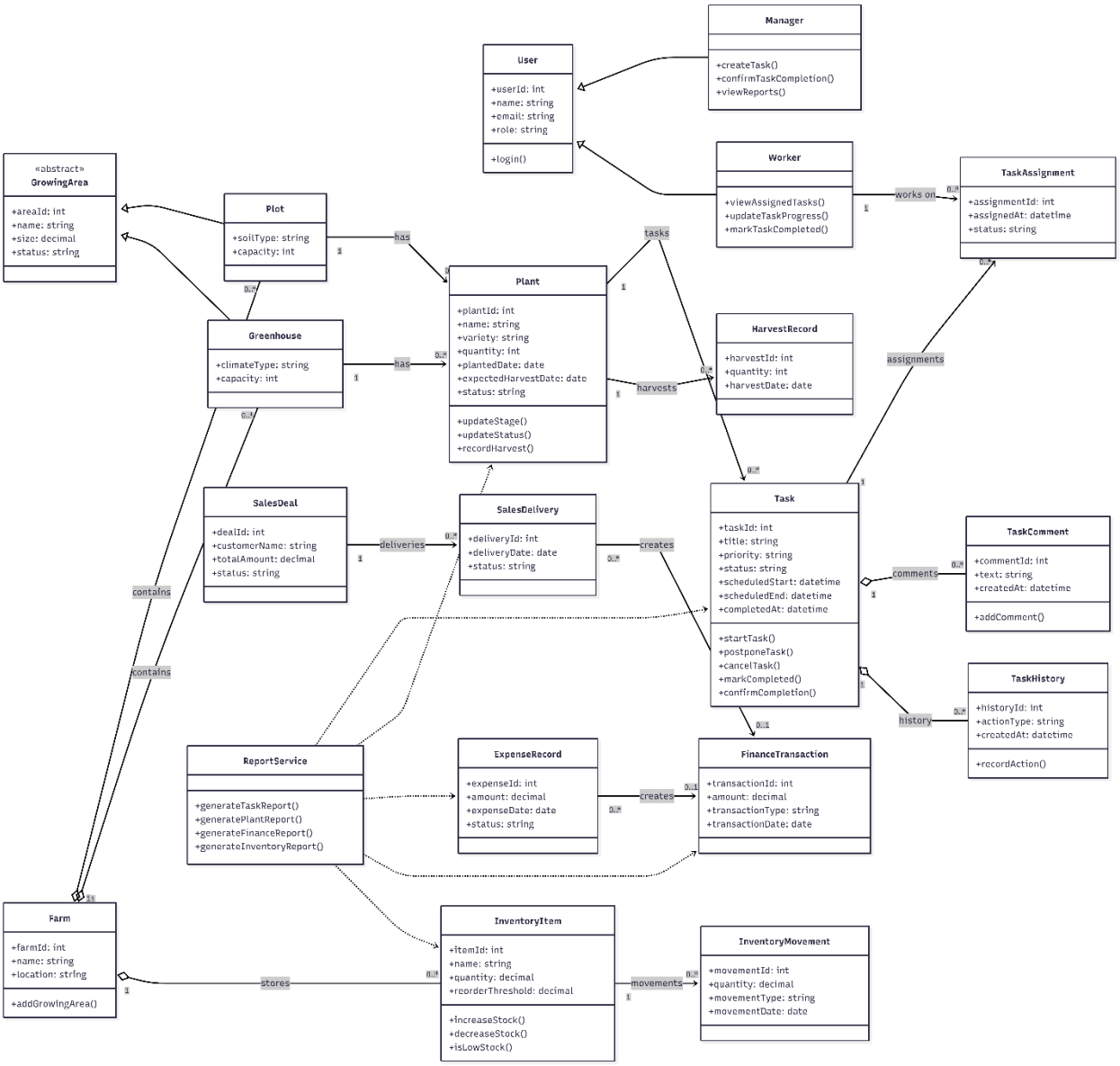
The Component Diagram should show that:

- the **User Interface** depends on all major business components
- each business component reads from and writes to the **Shared Database**
- the **Reporting Component** depends on data produced by the other components
- the **Task Component** depends on plant data
- the **Finance Component** may depend on harvest, task, and resource data
- the **Inventory Component** may be used by the task and finance modules for stock updates

2.5 Consistency Note

The component names should remain consistent with the database-backed domains used throughout the rest of the design section.

3. Class Diagram



3.1 Overview

The Class Diagram should model the main object-oriented structure of the system based on the existing relational database design. The classes should represent the core business entities, their attributes, their main operations, and their relationships. The diagram must include inheritance, association, aggregation, and dependency relationships, with multiplicities shown on all major links. The structure below is designed directly from the schema and is suitable for a class diagram with more than the minimum required 8 classes.

3.2 Main Classes to Include

1. User

Attributes

- id: Long
- fullName: String
- email: String
- passwordHash: String
- role: String
- isActive: Boolean
- createdAt: DateTime
- updatedAt: DateTime

Methods

- login()
- updateProfile()
- deactivate()
- reactivate()

2. Manager

Inheritance

- inherits from User

Methods

- createTask()
- assignWorker()
- confirmTaskCompletion()
- recordExpense()
- recordSaleDelivery()
- viewReports()

3. Worker

Inheritance

- inherits from User

Methods

- viewAssignedTasks()
- startTask()
- updateTaskProgress()
- markTaskCompleted()
- addTaskComment()
- recordResourceUsage()

4. Farm

Attributes

- id: Long
- name: String
- description: String
- locationText: String
- createdAt: DateTime
- updatedAt: DateTime

Methods

- addGrowingArea()
- updateFarmDetails()
- getFarmSummary()

5. GrowingArea

Attributes

- id: Long
- name: String

- code: String
- sizeValue: Decimal
- sizeUnit: String
- status: String
- notes: String

Methods

- updateArea()
- archiveArea()
- getAreaStatus()

This class should be used as a general parent for the area types.

6. Plot

Inheritance

- inherits from GrowingArea

Attributes

- soilType: String
- irrigationType: String

Methods

- assignPlant()
- clearPlot()

7. Greenhouse

Inheritance

- inherits from GrowingArea

Attributes

- greenhouseType: String
- temperatureMinC: Decimal
- temperatureMaxC: Decimal

- humidityTargetPercent: Decimal

Methods

- addPlant()
- removePlant()
- updateClimateTargets()

8. Plant

Attributes

- id: Long
- name: String
- variety: String
- quantity: Decimal
- quantityUnit: String
- plantedDate: Date
- expectedHarvestDate: Date
- status: String
- notes: String

Methods

- updateStage()
- updateStatus()
- recordHarvest()
- recordResourceUsage()

9. Task

Attributes

- id: Long
- title: String
- description: String

- category: String
- priority: String
- status: String
- scheduledStartAt: DateTime
- scheduledEndAt: DateTime
- actualStartAt: DateTime
- workerCompletedAt: DateTime
- managerConfirmedAt: DateTime

Methods

- startTask()
- postponeTask()
- cancelTask()
- markCompleted()
- confirmCompletion()
- addComment()

10. TaskAssignment

Attributes

- id: Long
- assignedAt: DateTime

Methods

- assignWorker()
- removeWorker()

This class links Task and Worker.

11. TaskComment

Attributes

- id: Long

- commentType: String
- message: String
- createdAt: DateTime

Methods

- addComment()
- editComment()

12. TaskHistory

Attributes

- id: Long
- actionType: String
- fieldName: String
- oldValue: String
- newValue: String
- actionNote: String
- createdAt: DateTime

Methods

- recordChange()
- getHistoryTimeline()

13. HarvestRecord

Attributes

- id: Long
- harvestedAt: DateTime
- quantity: Decimal
- quantityUnit: String
- qualityGrade: String
- notes: String

Methods

- saveHarvest()
- updateHarvestDetails()

14. ExpenseRecord

Attributes

- id: Long
- title: String
- description: String
- expenseKind: String
- amount: Decimal
- expenseDate: Date
- dueDate: Date
- paymentStatus: String

Methods

- createExpense()
- updatePaymentStatus()
- linkToTask()
- linkToPlant()

15. SalesDeal

Attributes

- id: Long
- title: String
- productName: String
- agreedQuantity: Decimal
- quantityUnit: String
- frequency: String

- intervalValue: Integer
- unitPrice: Decimal
- startDate: Date
- endDate: Date
- status: String

Methods

- createDeal()
- updateDeal()
- cancelDeal()

16. SalesDelivery

Attributes

- id: Long
- scheduledDate: Date
- deliveredDate: Date
- quantityDelivered: Decimal
- quantityUnit: String
- unitPrice: Decimal
- totalAmount: Decimal
- paymentStatus: String

Methods

- recordDelivery()
- updatePaymentStatus()

17. FinanceTransaction

Attributes

- id: Long
- transactionType: String

- sourceType: String
- transactionDate: Date
- amount: Decimal
- paymentMethod: String
- note: String

Methods

- createIncome()
- createExpense()
- getTransactionSummary()

18. InventoryItem

Attributes

- id: Long
- name: String
- unit: String
- currentQuantity: Decimal
- lowStockThreshold: Decimal
- storageLocation: String
- status: String
- notes: String

Methods

- increaseStock()
- decreaseStock()
- adjustStock()
- isLowStock()

19. InventoryMovement

Attributes

- id: Long
- movementType: String
- quantity: Decimal
- movementDate: DateTime
- note: String

Methods

- recordMovement()
- classifyMovement()

20. ReportService

Attributes

- reportType: String

Methods

- generateTaskReport()
- generatePlantReport()
- generateFinanceReport()
- generateInventoryReport()

This class is useful for showing dependency relationships in the class diagram.

3.3 Required Relationships

Inheritance

The diagram should include at least these inheritance relationships:

- Manager inherits from User
- Worker inherits from User
- Plot inherits from GrowingArea
- Greenhouse inherits from GrowingArea

Associations

The diagram should include these key associations:

- Farm — GrowingArea
- Plant — Task
- Task — TaskAssignment
- Worker — TaskAssignment
- Task — TaskComment
- Task — TaskHistory
- Plant — HarvestRecord
- SalesDeal — SalesDelivery
- InventoryItem — InventoryMovement

Aggregation

The diagram should show aggregation where the parent object conceptually contains related objects:

- Farm aggregates Plot
- Farm aggregates Greenhouse
- Farm aggregates InventoryItem
- Task aggregates TaskComment
- Task aggregates TaskHistory

Dependency

The diagram should show dependencies such as:

- ReportService depends on Task
- ReportService depends on Plant
- ReportService depends on ExpenseRecord
- ReportService depends on FinanceTransaction
- ReportService depends on InventoryItem

3.4 Multiplicities to Show

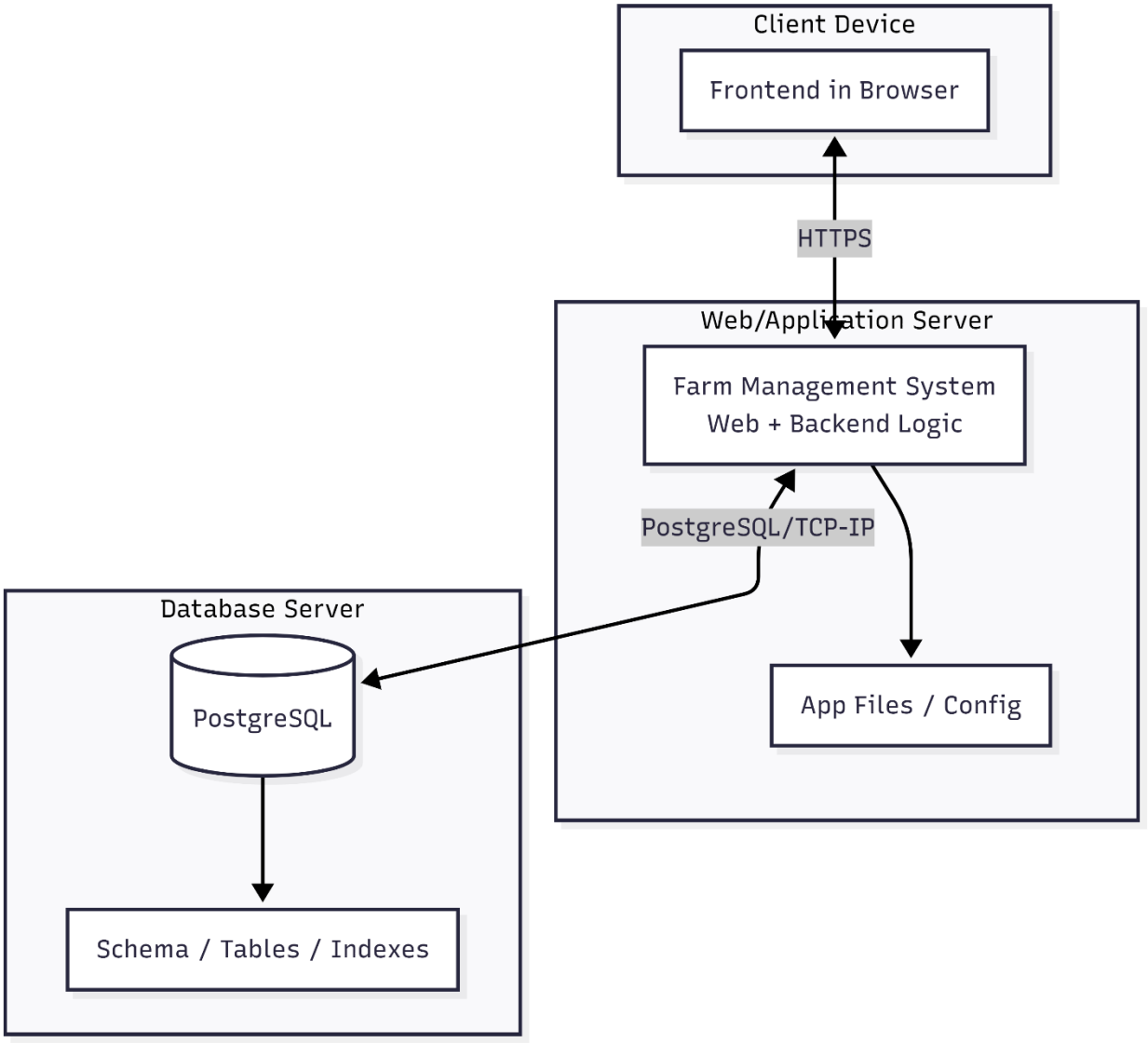
The main multiplicities should be:

- Farm 1 to Plot 0..*
- Farm 1 to Greenhouse 0..*
- Plot 1 to Plant 0..1
- Greenhouse 1 to Plant 0..*
- Plant 1 to Task 0..*
- Task 1 to TaskAssignment 0..*
- Worker 1 to TaskAssignment 0..*
- Task 1 to TaskComment 0..*
- Task 1 to TaskHistory 0..*
- Plant 1 to HarvestRecord 0..*
- SalesDeal 1 to SalesDelivery 0..*
- ExpenseRecord 1 to FinanceTransaction 0..1
- SalesDelivery 1 to FinanceTransaction 0..1
- InventoryItem 1 to InventoryMovement 0..*

3.5 Consistency Note

The classes and relationships above are based directly on the database structure and should remain aligned with the rest of the diagrams in the project.

4. Deployment Diagram



4.1 Overview

The Deployment Diagram should represent the physical deployment of the Farm Management System as a simple web-based application. Since the system is best modeled as a layered modular monolith, the deployment should show one main application connected to a PostgreSQL database. The goal is to keep the deployment realistic, simple, and consistent with the project scale.

4.2 Main Nodes

Client Device Node

This node represents:

- desktop browser
- laptop browser
- possibly mobile browser

Main responsibility:

- display the user interface
- send user actions to the application
- receive views, forms, and results

Artifact deployed:

- browser-rendered frontend interface

Web/Application Server Node

This node represents the main deployed application.

Main responsibility:

- process business logic
- authenticate users
- handle plant, task, finance, and inventory operations
- generate reports
- communicate with the database

Artifacts deployed:

- frontend web application bundle
- backend application service
- compiled source code / executable application
- configuration files

Database Server Node

This node represents the persistent data layer.

Main responsibility:

- store all application data
- enforce relationships and constraints
- support queries for all modules

Artifacts deployed:

- PostgreSQL database
- database schema
- tables, indexes, and constraints
- seed/reference data where needed

4.3 Communication Paths and Protocols

The Deployment Diagram should show these main communication links:

- **Client Device ↔ Web/Application Server**
  - protocol: **HTTPS**
  - purpose: login, page requests, form submission, API calls, report requests
- **Web/Application Server ↔ Database Server**
  - protocol: **PostgreSQL / TCP-IP**
  - purpose: reading and writing persistent application data

4.4 Deployment Notes

The deployment should reflect a simple and realistic system setup:

- one main application instance
- one relational database
- browser-based access for managers and workers

- no separate microservices
- no external actor systems directly integrated into the application

This keeps the deployment aligned with the chosen modular monolith architecture and the shared relational schema.

5. Wireframe Appendix

5.1 Overview

The wireframe appendix should contain a minimum of four annotated screens. These wireframes should reflect the major functional areas shown in the database design: login and access control, farm and plant management, task execution, finance/sales/inventory management, and reporting. The screens should remain simple and practical, since the project is a small management system rather than a commercial product.

5.2 Screen 1 – Login Screen

Purpose

This screen allows managers and workers to access the system.

Main elements

- email input
- password input
- login button
- error message area
- simple branding/header

Annotation points

- the login form authenticates against the users table
- user role determines available navigation after login
- inactive users should be blocked from access

Navigation flow

- successful login → Dashboard
- failed login → remain on Login screen with error message

5.3 Screen 2 – Dashboard Screen

Purpose

This screen acts as the main landing page after login and gives a quick overview of the system.

Main elements

- top navigation bar
- quick summary cards
- recent tasks panel
- plant/harvest summary panel
- finance summary panel
- inventory alert panel
- shortcuts to main modules

Annotation points

- task summary reads from tasks
- plant summary reads from plants and harvest_history
- finance summary reads from expense_records, sales_deliveries, and finance_transactions
- inventory alert section reads from inventory_items

Navigation flow

- Dashboard → Plants and Growing Areas
- Dashboard → Tasks
- Dashboard → Finance and Sales
- Dashboard → Inventory

5.4 Screen 3 – Plants and Growing Areas Screen

Purpose

This screen allows the manager to manage farm structure and plant records.

Main elements

- farm selector
- tabs or sections for Plots and Greenhouses
- plant list/table
- add/edit plant button
- status filter
- area detail panel
- plant detail panel

Annotation points

- farm data comes from farms
- plot data comes from plots
- greenhouse data comes from greenhouses
- plant stage and status come from plants and plant_stages
- selected plant details should show quantity, variety, stage, and expected harvest date

Navigation flow

- Dashboard → Plants and Growing Areas
- Plants and Growing Areas → Task creation for selected plant
- Plants and Growing Areas → Harvest record entry
- Plants and Growing Areas → back to Dashboard

5.5 Screen 4 – Task Management Screen

Purpose

This screen supports task management for managers and task execution for workers.

Main elements

- task list/table
- filters by status, priority, worker, and plant
- task detail side panel or detail page
- create task button
- assign worker action
- task comments section
- task history section
- resource usage entry section

Annotation points

- recurring data comes from recurring_task_series
- task rows come from tasks
- worker assignment data comes from task_assignments
- comments come from task_comments
- history comes from task_history
- resource usage comes from resource_usage

Navigation flow

- Dashboard → Tasks
- Tasks → Task detail
- Tasks → Create task form
- Task detail → Plant detail
- Task detail → back to Tasks list

5.6 Screen 5 – Finance, Sales, and Inventory Screen

Purpose

This screen manages expenses, sales records, financial transactions, and stock control in one admin-focused area, usually through tabs or sub-sections.

Main elements

- tab bar: Expenses / Sales / Inventory
- expense record table
- sales deal and sales delivery table
- finance summary cards
- inventory item list
- stock movement log
- add expense button
- add sales delivery button
- adjust stock button

Annotation points

- expense data comes from expense_records, expense_categories, and recurring_expenses
- sales data comes from sales_deals and sales_deliveries
- financial summaries come from finance_transactions
- stock data comes from inventory_items, inventory_categories, and inventory_movements

Navigation flow

- Dashboard → Finance, Sales, and Inventory
- Finance tab → Create Expense Record
- Sales tab → Record Sale Delivery
- Inventory tab → Adjust Stock / View Movements
- Finance, Sales, and Inventory → Reports

5.7 Navigation Flow Summary

The wireframes should show a simple navigation flow:

- **Login → Dashboard**
- **Dashboard → Plants and Growing Areas**
- **Dashboard → Tasks**
- **Dashboard → Finance, Sales, and Inventory**
- **Dashboard → Reports**
- **Plants and Growing Areas → Task creation / Harvest entry**
- **Tasks → Task detail / Plant detail**
- **Finance, Sales, and Inventory → Expense / Sales / Stock actions**

This navigation flow keeps the interface aligned with the same modules already defined in the component and database design.